In [150…
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,plot_tree
from sklearn.metrics import accuracy_score,classification_report,confusion_mat
import seaborn as sns
```

In [117…
```
#Loading the dataset
df=pd.DataFrame(pd.read_csv('Admission_Predict - Admission_Predict.csv'))
```

In [118…
```
#Shape For Dataset
df.shape
```

Out[118… (400, 9)

In [119…
```
df.head()
```

Out[119…

| | Serial No. | GRE Score | TOEFL Score | University Rating | SOP | LOR | CGPA | Research | Chance of Admit |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 337 | 118 | 4 | 4.5 | 4.5 | 9.65 | 1 | 0.92 |
| **1** | 2 | 324 | 107 | 4 | 4.0 | 4.5 | 8.87 | 1 | 0.76 |
| **2** | 3 | 316 | 104 | 3 | 3.0 | 3.5 | 8.00 | 1 | 0.72 |
| **3** | 4 | 322 | 110 | 3 | 3.5 | 2.5 | 8.67 | 1 | 0.80 |
| **4** | 5 | 314 | 103 | 2 | 2.0 | 3.0 | 8.21 | 0 | 0.65 |

In [120…
```
#Datatypes in dataset
df.dtypes
```

Out[120…
```
Serial No.            int64
GRE Score             int64
TOEFL Score           int64
University Rating     int64
SOP                 float64
LOR                 float64
CGPA                float64
Research              int64
Chance of Admit     float64
dtype: object
```

In [121…
```
#Describing the data
df.describe()
```

Out[121…

| | Serial No. | GRE Score | TOEFL Score | University Rating | SOP | LOR |
|---|---|---|---|---|---|---|
| **count** | 400.000000 | 400.000000 | 400.000000 | 400.000000 | 400.000000 | 400.000000 |
| **mean** | 200.500000 | 316.807500 | 107.410000 | 3.087500 | 3.400000 | 3.452500 |
| **std** | 115.614301 | 11.473646 | 6.069514 | 1.143728 | 1.006869 | 0.898478 |
| **min** | 1.000000 | 290.000000 | 92.000000 | 1.000000 | 1.000000 | 1.000000 |
| **25%** | 100.750000 | 308.000000 | 103.000000 | 2.000000 | 2.500000 | 3.000000 |
| **50%** | 200.500000 | 317.000000 | 107.000000 | 3.000000 | 3.500000 | 3.500000 |
| **75%** | 300.250000 | 325.000000 | 112.000000 | 4.000000 | 4.000000 | 4.000000 |
| **max** | 400.000000 | 340.000000 | 120.000000 | 5.000000 | 5.000000 | 5.000000 |

In [122…
```
#number of zeros
(df==0).sum()
```

Out[122…
```
Serial No.            0
GRE Score             0
TOEFL Score           0
University Rating     0
SOP                   0
LOR                   0
CGPA                  0
Research            181
Chance of Admit       0
dtype: int64
```

In [123…
```
#Check for Null Values
(df.isnull()).sum()
```

Out[123…
```
Serial No.          0
GRE Score           0
TOEFL Score         0
University Rating   0
SOP                 0
LOR                 0
CGPA                0
Research            0
Chance of Admit     0
dtype: int64
```

In [124…
```
df=df.drop(columns=['Serial No.'])
```

In [125…
```
df.mean()
```

Out[125…
```
GRE Score            316.807500
TOEFL Score          107.410000
University Rating      3.087500
SOP                    3.400000
LOR                    3.452500
CGPA                   8.598925
Research               0.547500
Chance of Admit        0.724350
dtype: float64
```

In [126…
```
df.median()
```

Out[126…
```
GRE Score            317.00
TOEFL Score          107.00
University Rating      3.00
SOP                    3.50
LOR                    3.50
CGPA                   8.61
Research               1.00
Chance of Admit        0.73
dtype: float64
```

In [127…
```
df.mode()
```

Out[127…

| | GRE Score | TOEFL Score | University Rating | SOP | LOR | CGPA | Research | Chance of Admit |
|---|---|---|---|---|---|---|---|---|
| **0** | 312 | 110.0 | 3.0 | 3.5 | 3.0 | 8.0 | 1.0 | 0.64 |
| **1** | 324 | NaN | NaN | 4.0 | NaN | NaN | NaN | NaN |

In [128…
```
df.std()
```

Out[128…
```
GRE Score            11.473646
TOEFL Score           6.069514
University Rating     1.143728
SOP                   1.006869
LOR                   0.898478
CGPA                  0.596317
Research              0.498362
Chance of Admit       0.142609
dtype: float64
```

In [129…
```
x=df[['GRE Score', 'TOEFL Score', 'University Rating',
          'SOP', 'LOR', 'CGPA', 'Research']]
```

In [130…
```
x
```

Out[130…

| | GRE Score | TOEFL Score | University Rating | SOP | LOR | CGPA | Research |
|---|---|---|---|---|---|---|---|
| **0** | 337 | 118 | 4 | 4.5 | 4.5 | 9.65 | 1 |
| **1** | 324 | 107 | 4 | 4.0 | 4.5 | 8.87 | 1 |
| **2** | 316 | 104 | 3 | 3.0 | 3.5 | 8.00 | 1 |
| **3** | 322 | 110 | 3 | 3.5 | 2.5 | 8.67 | 1 |
| **4** | 314 | 103 | 2 | 2.0 | 3.0 | 8.21 | 0 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **395** | 324 | 110 | 3 | 3.5 | 3.5 | 9.04 | 1 |
| **396** | 325 | 107 | 3 | 3.0 | 3.5 | 9.11 | 1 |
| **397** | 330 | 116 | 4 | 5.0 | 4.5 | 9.45 | 1 |
| **398** | 312 | 103 | 3 | 3.5 | 4.0 | 8.78 | 0 |
| **399** | 333 | 117 | 4 | 5.0 | 4.0 | 9.66 | 1 |

400 rows × 7 columns

In [131…
```
y=(df['Chance of Admit']>=0.8).astype(int)
y
```

Out[131…
```
0      1
1      0
2      0
3      1
4      0
      ..
395    1
396    1
397    1
398    0
399    1
Name: Chance of Admit, Length: 400, dtype: int64
```

In [132…
```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_stat
```

In [133…
```
#Default value for finding the impurity is Gini index
tree=DecisionTreeClassifier()
```

In [134…
```
tree.fit(x_train,y_train)
```

Out[134…

▾ DecisionTreeClassifier

```
DecisionTreeClassifier()
```

In [135…
```
y_pred=tree.predict(x_test)
```

In [136…
```
y_pred
```

Out[136…
```
array([1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0])
```

In [137…
```
accuracy_score(y_test,y_pred)
```

Out[137…
```
0.875
```

In [138…
```
confusion_matrix(y_test,y_pred)
```

Out[138…
```
array([[44,  4],
       [ 6, 26]])
```

In [139…
```
print(classification_report(y_test,y_pred))
```

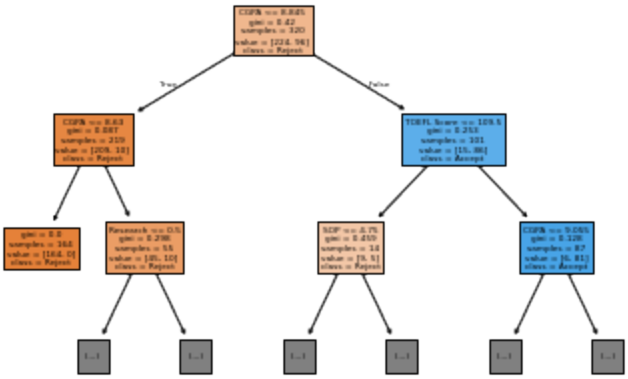
```
              precision    recall  f1-score   support

           0       0.88      0.92      0.90        48
           1       0.87      0.81      0.84        32

    accuracy                           0.88        80
   macro avg       0.87      0.86      0.87        80
weighted avg       0.87      0.88      0.87        80
```

In [140…
```
plot_tree(tree,max_depth=2,feature_names=['GRE Score', 'TOEFL Score', 'Univers
```

Out[140…
```
[Text(0.4230769230769231, 0.875, 'CGPA <= 8.845\ngini = 0.42\nsamples = 320\n
value = [224, 96]\nclass = Reject'),
 Text(0.15384615384615385, 0.625, 'CGPA <= 8.63\ngini = 0.087\nsamples = 219\
nvalue = [209, 10]\nclass = Reject'),
 Text(0.28846153846153844, 0.75, 'True  '),
 Text(0.07692307692307693, 0.375, 'gini = 0.0\nsamples = 164\nvalue = [164,
0]\nclass = Reject'),
 Text(0.23076923076923078, 0.375, 'Research <= 0.5\ngini = 0.298\nsamples = 5
5\nvalue = [45, 10]\nclass = Reject'),
 Text(0.15384615384615385, 0.125, '\n  (...)  \n'),
 Text(0.3076923076923077, 0.125, '\n  (...)  \n'),
 Text(0.6923076923076923, 0.625, 'TOEFL Score <= 109.5\ngini = 0.253\nsamples
= 101\nvalue = [15, 86]\nclass = Accept'),
 Text(0.5576923076923077, 0.75, '  False'),
 Text(0.5384615384615384, 0.375, 'SOP <= 4.75\ngini = 0.459\nsamples = 14\nva
lue = [9, 5]\nclass = Reject'),
 Text(0.46153846153846156, 0.125, '\n  (...)  \n'),
 Text(0.6153846153846154, 0.125, '\n  (...)  \n'),
 Text(0.8461538461538461, 0.375, 'CGPA <= 9.055\ngini = 0.128\nsamples = 87\n
value = [6, 81]\nclass = Accept'),
 Text(0.7692307692307693, 0.125, '\n  (...)  \n'),
 Text(0.9230769230769231, 0.125, '\n  (...)  \n')]
```



In [147…
```
Gre_Score = int(input('GRE Score '))
TOEFL_Score = float(input('Enter TOEFL Score '))
uni_rating = float(input('Enter University Rating'))
sop = float(input('Enter SOP '))
lor = float(input('LOR'))
cgpa = float(input('CGPA'))
research = int(input('Research'))
chance=tree.predict([[Gre_Score,TOEFL_Score,uni_rating,sop,lor,cgpa,research]]
print(chance)
if(chance==1):
    print("Congratulations you are through")
else:
    print("Better luck next time")
```

```
GRE Score 320
Enter TOEFL Score 100
Enter University Rating1
Enter SOP 5.0
LOR5.0
CGPA9.99
Research1
1
Congratulations you are through
```

```
/home/student/anaconda3/lib/python3.9/site-packages/sklearn/base.py:493: UserWa
rning: X does not have valid feature names, but DecisionTreeClassifier was fitt
ed with feature names
  warnings.warn(
```

In [142…
```
tree2=DecisionTreeClassifier(criterion="entropy")
```

In [143…
```
tree2.fit(x_train,y_train)
```

Out[143…
```
DecisionTreeClassifier(criterion='entropy')
```

In [144…
```
y_pred2=tree2.predict(x_test)
```

In [145…
```
y_pred2
```

Out[145…
```
array([1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0,
       1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0,
       1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0])
```

In [146…
```
accuracy_score(y_test,y_pred2)
```

Out[146…
```
0.85
```

In [106…
```
confusion_matrix(y_test,y_pred2)
```

Out[106…
```
array([[42,  6],
       [ 5, 27]])
```

In [107…
```
print(classification_report(y_test,y_pred2))
```

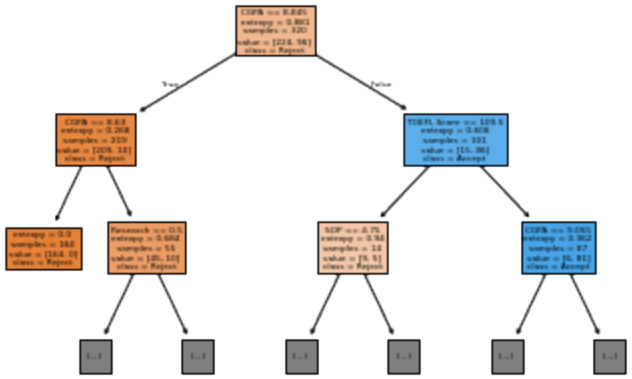
```
              precision    recall  f1-score   support

           0       0.89      0.88      0.88        48
           1       0.82      0.84      0.83        32

    accuracy                           0.86        80
   macro avg       0.86      0.86      0.86        80
weighted avg       0.86      0.86      0.86        80
```

In [108…
```
plot_tree(tree2,max_depth=2,feature_names=['GRE Score', 'TOEFL Score', 'Univer
          'SOP', 'LOR', 'CGPA', 'Research'],class_names=['Reject','Accept'],fi
```

Out[108…
```
[Text(0.4230769230769231, 0.875, 'CGPA <= 8.845\nentropy = 0.881\nsamples = 3
20\nvalue = [224, 96]\nclass = Reject'),
 Text(0.15384615384615385, 0.625, 'CGPA <= 8.63\nentropy = 0.268\nsamples = 2
19\nvalue = [209, 10]\nclass = Reject'),
 Text(0.28846153846153844, 0.75, 'True  '),
 Text(0.07692307692307693, 0.375, 'entropy = 0.0\nsamples = 164\nvalue = [16
4, 0]\nclass = Reject'),
 Text(0.23076923076923078, 0.375, 'Research <= 0.5\nentropy = 0.684\nsamples
= 55\nvalue = [45, 10]\nclass = Reject'),
 Text(0.15384615384615385, 0.125, '\n  (...)  \n'),
 Text(0.3076923076923077, 0.125, '\n  (...)  \n'),
 Text(0.6923076923076923, 0.625, 'TOEFL Score <= 109.5\nentropy = 0.606\nsamp
les = 101\nvalue = [15, 86]\nclass = Accept'),
 Text(0.5576923076923077, 0.75, '  False'),
 Text(0.5384615384615384, 0.375, 'SOP <= 4.75\nentropy = 0.94\nsamples = 14\n
value = [9, 5]\nclass = Reject'),
 Text(0.46153846153846156, 0.125, '\n  (...)  \n'),
 Text(0.6153846153846154, 0.125, '\n  (...)  \n'),
 Text(0.8461538461538461, 0.375, 'CGPA <= 9.055\nentropy = 0.362\nsamples = 8
7\nvalue = [6, 81]\nclass = Accept'),
 Text(0.7692307692307693, 0.125, '\n  (...)  \n'),
 Text(0.9230769230769231, 0.125, '\n  (...)  \n')]
```



In [109…
```
Gre_Score = int(input('GRE Score '))
TOEFL_Score = float(input('Enter TOEFL Score '))
uni_rating = float(input('Enter University Rating'))
sop = float(input('Enter SOP '))
lor = float(input('LOR'))
cgpa = float(input('CGPA'))
research = int(input('Research'))
chance=tree2.predict([[Gre_Score,TOEFL_Score,uni_rating,sop,lor,cgpa,research]
print(chance)
if(chance==1):
    print("Congratulations you are through")
else:
    print("Better luck next time")
```

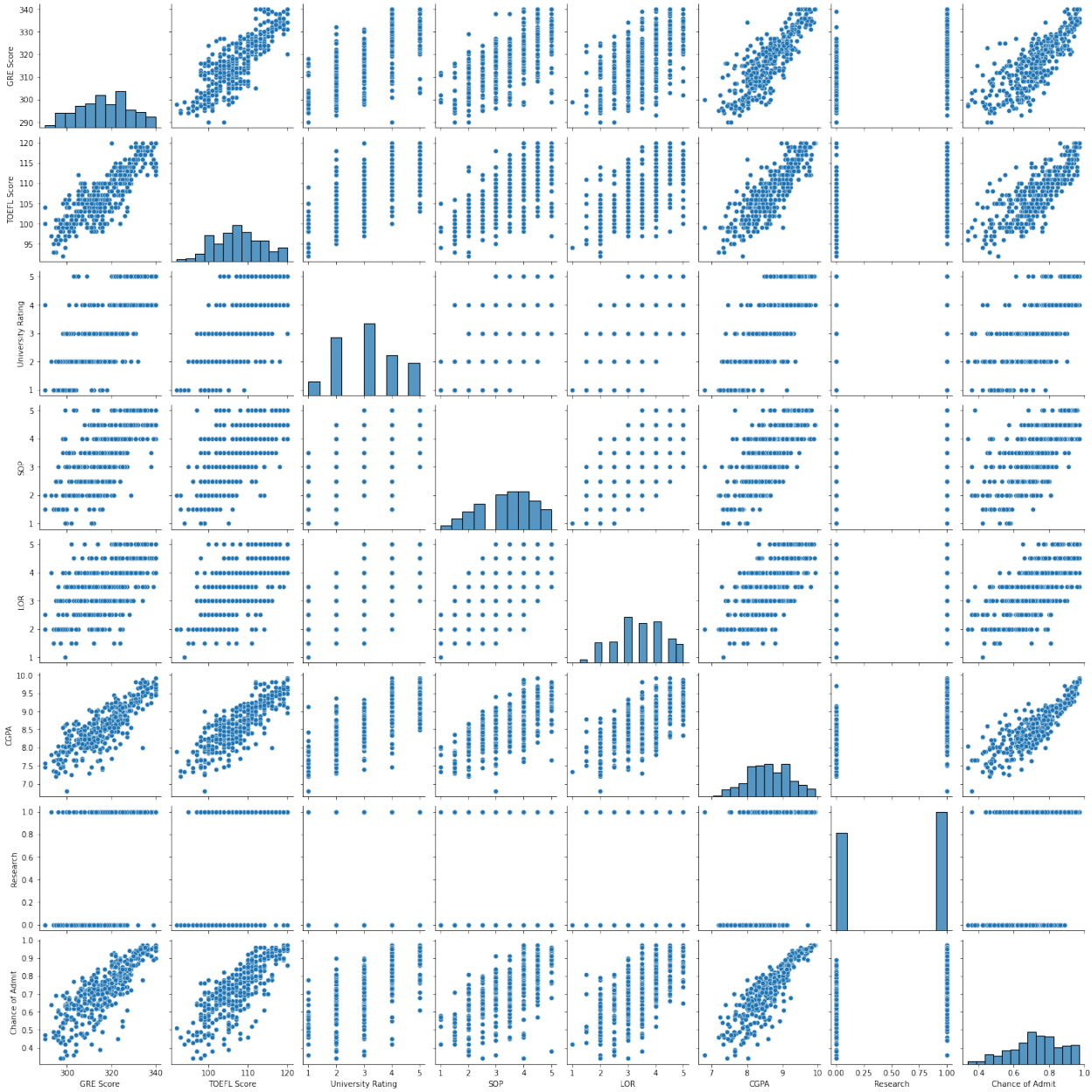
```
GRE Score 332
Enter TOEFL Score 100
Enter University Rating2
Enter SOP 5.0
LOR3.5
CGPA9.96
Research1
1
Congratulations you are through
```

```
/home/student/anaconda3/lib/python3.9/site-packages/sklearn/base.py:493: UserWa
rning: X does not have valid feature names, but DecisionTreeClassifier was fitt
ed with feature names
  warnings.warn(
```

In [110…
```
#Graphs
```

In [111…
```
sns.pairplot(df)
```

Out[111…
```
<seaborn.axisgrid.PairGrid at 0x7efd49f58eb0>
```

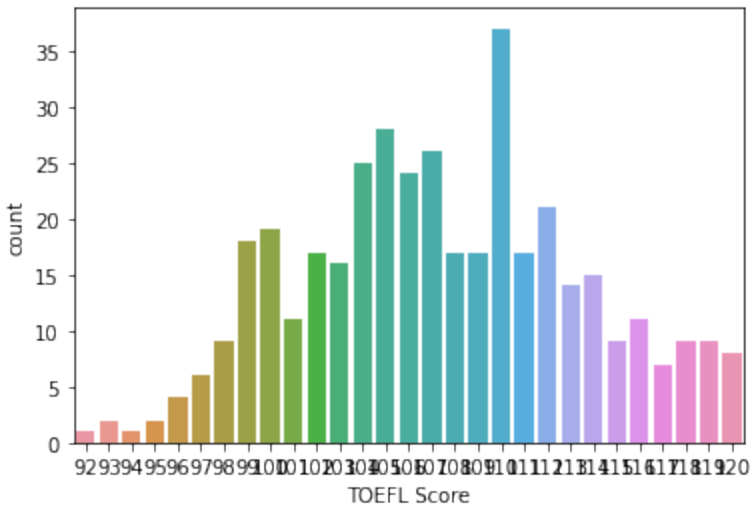



In [113…

```
count=plt.figure
sns.countplot(data=df,x=df['TOEFL Score'])
```

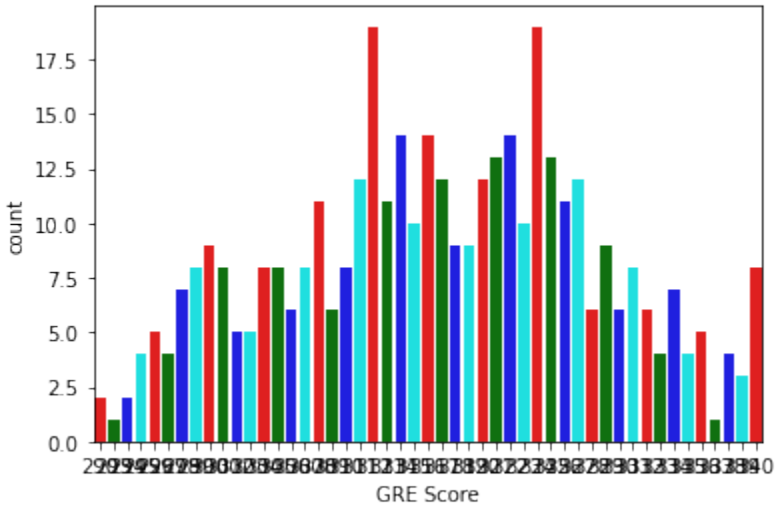
Out[113…

```
<AxesSubplot:xlabel='TOEFL Score', ylabel='count'>
```




In [148…
```
sns.countplot(data=df,x=df['GRE Score'],palette=['red','green','blue','cyan'])
```

Out[148…
```
<AxesSubplot:xlabel='GRE Score', ylabel='count'>
```




In [ ]: